

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	R. Uday Kumar Reddy	Reg. No.:	192424190
Section / Batch:	Slot-C	Date:	26/8/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q. Sudoku Variation with Diagonal Constraints (Backtracking / NP-Complete Problem)

Solve a Sudoku puzzle where both diagonals must also contain unique numbers. Clearly define additional diagonal constraints. Analyze how constraints interact with standard Sudoku rules. Develop a backtracking-based solution. Apply constraint propagation and pruning. Justify correctness and discuss increased complexity.

Code of Conduct

/ R. Uday Kumar Reddy (192424190)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R. Uday_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q27. Sudoku Variation with Diagonal Constraints (Backtracking / NP-Complete Problem)

Solve a Sudoku puzzle where both diagonals must also contain unique numbers. Clearly define additional diagonal constraints. Analyze how constraints interact with standard Sudoku rules. Develop a backtracking-based solution. Apply constraint propagation and pruning. Justify correctness and discuss increased complexity.

1. Problem Definition

A standard 9×9 Sudoku requires:

- Each row contains numbers 1–9 exactly once.
- Each column contains numbers 1–9 exactly once.
- Each 3×3 subgrid contains numbers 1–9 exactly once.

In the Diagonal Sudoku variation, two additional constraints are introduced:

- The main diagonal (top-left to bottom-right) must contain numbers 1–9 exactly once.
- The secondary diagonal (top-right to bottom-left) must contain numbers 1–9 exactly once.

Thus, a valid solution must satisfy four types of constraints:

1. Row constraint
2. Column constraint
3. 3×3 subgrid constraint
4. Main and secondary diagonal constraints

Standard Sudoku Constraints

In a standard Sudoku puzzle:

Row Constraint

Each row must contain the digits 1–9 exactly once.

Example:

1 2 3 4 5 6 7 8 9

No number can be repeated within the same row.

Column Constraint

Each column must contain the digits 1–9 exactly once.

1

2

3
4
5
6
7
8
9

No duplication is allowed in any column.

Subgrid Constraint

The board is divided into nine 3×3 subgrids.

Example:

1 2 3
4 5 6
7 8 9

Each subgrid must contain all digits from 1 to 9 exactly once.

Additional Diagonal Constraints

Diagonal Sudoku introduces two extra conditions.

Main Diagonal Constraint

The main diagonal runs from the top-left corner to the bottom-right corner.

Cells belonging to the main diagonal:

(0,0)
(1,1)
(2,2)
(3,3)
(4,4)
(5,5)
(6,6)
(7,7)
(8,8)

The numbers appearing on these cells must be:

1,2,3,4,5,6,7,8,9

without repetition.

Secondary Diagonal Constraint

The secondary diagonal runs from the top-right corner to the bottom-left corner.

Cells:

(0,8)

(1,7)

(2,6)

(3,5)

(4,4)

(5,3)

(6,2)

(7,1)

(8,0)

These cells must also contain digits 1–9 exactly once.

Special Case: Center Cell

The center cell (4,4) belongs to both diagonals.

Therefore, any value assigned to this position must satisfy:

- Row constraint
- Column constraint
- Subgrid constraint
- Main diagonal constraint
- Secondary diagonal constraint

This makes the center cell one of the most constrained positions in the puzzle.

Interaction Between Constraints

The additional diagonal rules interact with standard Sudoku constraints in several ways.

Suppose a cell lies on the main diagonal.

Before assigning a number, the algorithm must verify:

1. Number is not already present in the row.
2. Number is not already present in the column.

3. Number is not already present in the subgrid.
4. Number is not already present in the main diagonal.

If the cell lies on both diagonals, an additional check is required.

Thus the possible candidate values become:

$$\text{Candidates} = \text{RowCandidates} \cap \text{ColumnCandidates} \cap \text{BoxCandidates} \\ \cap \text{DiagonalCandidates}$$

The intersection operation significantly reduces possible choices.

For example:

Row Candidates = {1,2,4,6}

Column Candidates = {2,4,5,7}

Box Candidates = {1,2,4}

Diagonal Candidates = {2,4,8}

Result:

Possible Values = {2,4}

Instead of trying nine numbers, the solver needs to test only two values.

Backtracking Solution

Backtracking is a depth-first search technique that systematically explores possible assignments.

The main idea is:

1. Find an empty cell.
2. Determine all valid candidate values.
3. Place one candidate.
4. Recursively solve the remaining puzzle.
5. If a contradiction occurs, remove the value.
6. Try another candidate.
7. Continue until the puzzle is solved.

Algorithm Steps

Step 1: Locate Empty Cell

Search for the first unassigned cell.

Example:

5 3 0

6 0 0

0 9 8

Here "0" indicates an empty position.

Step 2: Generate Candidate Values

Determine which numbers can legally occupy the empty cell.

Candidate generation considers:

- Row restrictions
- Column restrictions
- Box restrictions
- Diagonal restrictions

Step 3: Assign Value

Choose a valid candidate and place it in the cell.

Before:

5 3 0

After:

5 3 4

Step 4: Recursive Call

Attempt to solve the rest of the puzzle.

Solve(next_empty_cell)

Step 5: Backtrack if Necessary

If no valid number exists:

5 3 4

6 0 0

Assume later a contradiction appears.

Remove 4:

5 3 0

6 0 0

Try another candidate.

Pseudocode

Solve(Board)

If puzzle complete

 Return TRUE

Find empty cell

For num = 1 to 9

 If Valid(Board,row,col,num)

 Place num

 If Solve(Board)

 Return TRUE

 Remove num

Return FALSE

Constraint Propagation

Constraint propagation reduces the search space before backtracking occurs.

The goal is to eliminate impossible values early.

Instead of guessing blindly, the solver continuously updates possible candidates.

Example:

Cell (3,3)

Possible:

{1,2,3,4,5,6,7,8,9}

After row analysis:

{2,4,5,7}

After column analysis:

{2,4,7}

After box analysis:

{2,7}

After diagonal analysis:

{7}

Since only one value remains, the cell must contain 7.

This avoids unnecessary recursion.

Pruning Techniques

1. Forward Checking

Whenever a value is assigned:

Cell = 5

remove 5 from the candidate lists of all related cells.

This prevents invalid assignments later.

2. Minimum Remaining Value (MRV)

Choose the cell with the fewest candidate values.

Example:

Cell A = {2}

Cell B = {1,4,5}

Cell C = {3,7}

MRV selects Cell A first.

This often leads to faster solutions.

3. Degree Heuristic

Choose the variable that affects the largest number of unassigned cells.

This helps maximize future constraint propagation.

4. Naked Singles

If a cell has only one possible value:

{8}

Assign it immediately.

5. Hidden Singles

If a number can appear in only one position within a row, column, box, or diagonal, place it there directly.

Correctness Proof

The algorithm is correct because it maintains all Sudoku constraints at every step.

Validity

A number is inserted only if:

- Row constraint holds.
- Column constraint holds.
- Subgrid constraint holds.

- Main diagonal constraint holds.
- Secondary diagonal constraint holds.

Therefore, every partial solution remains valid.

Completeness

Backtracking explores every legal assignment.

If a solution exists, eventually one search path reaches it.

Hence the algorithm is complete.

Termination

The puzzle contains a finite number of cells (81).

Each recursive call fills at least one cell.

Eventually:

- Solution found, or
- All possibilities exhausted.

Thus the algorithm always terminates.

Complexity Analysis

Let:

$$n = 81$$

represent the number of cells.

Each cell can potentially take:

$$9$$

values.

Worst-case search:

$$O(9^{81})$$

which is exponential.

Since Sudoku is NP-Complete, exponential behavior is expected in the worst case.

Cost of Constraint Checking

For every assignment:

Row Check

$$O(9)$$

Column Check

$$O(9)$$

Box Check

$$O(9)$$

Diagonal Check

$$O(9)$$

Total:

$$O(36)$$

which simplifies to:

$$O(1)$$

because the board size is fixed.

Effect of Diagonal Constraints

Diagonal constraints add extra checking overhead.

However, they also eliminate many invalid assignments.

Therefore:

Advantages

- Smaller search space
- Stronger pruning
- Faster practical solving

Disadvantages

- More constraint checks
- Increased implementation complexity

In practice, the reduction in search space outweighs the additional checking cost.

Applications

Diagonal Sudoku solving techniques are used in:

- Artificial Intelligence
- Constraint Satisfaction Problems (CSP)
- Automated Puzzle Solvers
- Logic Programming
- Scheduling Systems
- Resource Allocation Problems
- Game AI
- Search Optimization Algorithms

The same principles are applied in real-world planning and optimization tasks.

Code:

```
def print_board(board):
    for row in board:
        print(" ".join(map(str, row)))

def find_empty(board):
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                return row, col
    return None

def valid_standard(board, row, col, num):
    for j in range(9):
        if board[row][j] == num:
            return False
    for i in range(9):
        if board[i][col] == num:
            return False
    start_row = (row // 3) * 3
```

```

start_col = (col // 3) * 3
for i in range(start_row, start_row + 3):
    for j in range(start_col, start_col + 3):
        if board[i][j] == num:
            return False
    return True
def backtrack_standard(board):
    empty = find_empty(board)
    if empty is None:
        return True
    row, col = empty
    for num in range(1, 10):
        if valid_standard(board, row, col, num):
            board[row][col] = num
            if backtrack_standard(board):
                return True
            board[row][col] = 0
    return False
def valid_diagonal(board, row, col, num):
    if not valid_standard(board, row, col, num):
        return False
    if row == col:
        for i in range(9):
            if board[i][i] == num:
                return False
    if row + col == 8:
        for i in range(9):
            if board[i][8 - i] == num:
                return False
    return True

```

```

def backtrack_diagonal(board):
    empty = find_empty(board)
    if empty is None:
        return True
    row, col = empty
    for num in range(1, 10):
        if valid_diagonal(board, row, col, num):
            board[row][col] = num
            if backtrack_diagonal(board):
                return True
            board[row][col] = 0
    return False

puzzle = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],

    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],

    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

standard = [row[:] for row in puzzle]
print("STANDARD SUDOKU")
print("=====")
if backtrack_standard(standard):
    print_board(standard)

```

```

else:
    print("No solution")
diagonal = [row[:] for row in puzzle]
print("\nDIAGONAL SUDOKU")
print("=====")
if backtrack_diagonal(diagonal):
    print_board(diagonal)
else:
    print("No diagonal solution")

```

Output:

```

STANDARD SUDOKU
=====
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

DIAGONAL SUDOKU
=====
No diagonal solution

```

Conclusion

Diagonal Sudoku extends the traditional Sudoku puzzle by introducing uniqueness constraints on both diagonals. These additional requirements interact with row, column, and subgrid rules, making the puzzle more constrained and challenging. A backtracking algorithm systematically explores possible assignments, while constraint propagation and pruning techniques significantly reduce the search space. The solver remains correct because every assignment satisfies all constraints before proceeding. Although the problem remains NP-Complete and has exponential worst-case complexity, practical performance is greatly improved through heuristics such as MRV, forward checking, naked singles, and hidden singles. Thus, backtracking combined with intelligent constraint management provides an effective solution for solving diagonal Sudoku puzzles.